

1. Describe lexical and syntax analysis with a few sentences! 4.2&4.3

Lexical analysis: It is essentially a pattern matcher and serves as the front end of a syntax analyzer.

It collects lexemes (collects characters into logical groupings) from a given input string and produce the corresponding tokens (internal codes' categories).

Syntax analysis: the process of analyzing syntax.

It serves two purposes:

1. Make sure the syntax of the input program is syntactically correct.
2. If the grammar input is correct, it will generate a complete parse tree.

2. What are the primary tasks of a lexical analyzer? (3 points) 4.2 Lexical Analysis

(Describe it with your own words, what steps are performed and what produced by it)

Primary tasks: It helps to convert a sequence of characters into a sequence of tokens.

Steps: 1. It collects characters into logical groupings and assigns internal codes to the groupings according to their structure.

2. It extract lexemes from a given input string and produce the corresponding tokens.

3. It skipping comments and white space outside lexemes.

3. What is a reserved word? 5.2.3 Special Words

A reserved word is a special word of programming language which cannot be used as a name. In most languages, special words are classified as result words

4. What is a keyword? 5.2.3 Special Words

A keyword is a word that is special only in certain contexts

5. What is the l-value of a variable? What is the r-value? (2 marks) 5.3.2 Address

l-value: the address of this variable

r-value: the value of this variable

6. Describe the ways that aliases can occur with pass-by-reference parameters 5.3.2 Address

Aliases are two or more variables bound to the same memory location.

An example: `int a=0; int &a1=a;` In this case, a and a1 are aliases.

7. Write a simple assignment statement with one arithmetic operator in some language you know. For each component of the statement, list the various bindings that are required to determine the semantics when the statement is executed. For each binding, indicate the binding time used for the language. (4 marks)

5.4 The Concept of Binding

(C language) `int i=0; i=i+1;`

The type for i: bounded at compile time;

Set of possible values of i: bounded at compiler design time;

The meaning of the operator+: bounded at compile time;

The internal representation of literal 1 is bound at compiler design time;

The value of i: bounded at execution time;

8. Explain why dynamic type bindings is closely related to implicit heap-dynamic variables, using the following code example: 为什么动态绑定与隐式堆动态变量密切相关 5.4 The Concept of Binding

```
var a;  
...  
a = "Hello";  
...  
a = 123.456;
```

The variable `a` is bounded at runtime, in this case, firstly `a` is "Hello" and then it changes to "123.456". And implicit heap-dynamic variables are bounded to heap storage only when they are assigned values, which is matched with dynamic type bindings.

9. Describe the case when a variable does not have a name 5.4.3.3 Explicit Heap- Dynamic Variables

Explicit heap-dynamic variables are nameless (abstract) memory cells. They often do not have identifiers associated with them. They can be referenced only by pointer or reference type variables.

10. Consider the following C program, which variable has the longest scope? 5.5 Scope

Explain your answer with details! [3 marks]

```
int a;  
int main()  
{  
    int b;  
    // ..  
    // ..  
}  
int c;
```

`a` has the longest scope.

`a` is accessible in all time in this process, `b` is limited in `main()`, `c` is accessible only after declaration in the last line.

11. 5.5 Scope

Explain C's scoping and lifetime for all avg variables in the code fragment:

```
void sub(){  
    int avg;  
    ...  
    for( ; ; ){  
        int avg;  
        avg++  
        ...  
    }  
    ...  
}  
void printavg(){  
    int avg;  
    ...  
    sub();  
}
```

the first avg and third avg:

scope: all inside the function `sub()` / `printavg()`

lifetime: available until the function that calls it is done

the second avg:

scope: limited in the for loop

lifetime: available until the for loop is done

12. What is a static ancestor of a subprogram? Show an example (2 marks) 5.5.1 Static Scope

The static ancestor of a subprogram is its static parent and the static parent derived from it, until the largest one contains all the subprogram.

Example:

```
function() { //it is static ancestor of sub2
  function_sub1() {
    int x=1;
    function_sub2();
  }
  function_sub2() {
    int y=x;
  }
}
```

13. What is a block? 5.5.2 Blocks

It is the scope of local variables in a piece of code (this section of code between each pair of braces)

14. What is dynamic scoping and a dynamic ancestor of a subprogram? 5.5.6 Dynamic Scope

Show an example! (2 points)

Scope refers to the places in a program where a variable is visible and can be referenced.

Under **dynamic scoping**, a variable is bound to the most recent value assigned to that variable.

Dynamic ancestors of a subprogram are all the procedures called before it during the execution of a program.

Example:

```
func 1 ( ) // which is a dynamic ancestor of func_sub1 ( )
{
  func_sub1 ( ) {
    int i=1 ; } //it is a dynamic scoping
}
```

15. Define scope and lifetime. 5.6 Scope and Lifetime

Scope: The range of statements where the variable is visible.

Lifetime: The time when the variable is bound to a specific memory location.

16. What is the referencing environment of a statement? 5.7 Referencing Environments

The reference environment of a statement refers to the collection of all visible variables in this statement

17. 强制转换 Explain how the coercion rules of a language affect its error detection. (4 marks) 6.13 Strong Typing

It reduces the benefits of type checking.

Example: here are some codes:

```
int a, double b,c,d;
```

if the input should be: $d=b*c$; but we make a mistake: $d=b*a$; the compiler would not detect this as an error instead of coercing int a to double a.

18. What is type equivalence? 6.14 Type Equivalence

Two types are equivalent if an operand of one type in an expression can be substituted for one of the other types without coercion. It is a strict form of type compatibility.

19. How do parentheses affect the precedence rule? What will be the evaluation order? 7.2.1.3 Parentheses

we can alter the precedence and associativity rules by using parentheses in expressions.

A parenthesized part of an expression has precedence over its adjacent unparenthesized parts.

20. Define functional side effect, important to show examples as well. 7.2.2.1 Side Effects

If one function changes either its parameter or a global variable, **it has side effects**.

Here is a expression: $a + \text{fun}(a)$, if $\text{fun}(a)$ has side effects, when function change a , the order of evaluation of these two operands maybe changed.

21. Explain why operator overloading increases the semantic capability of a language? 7.3 Overloaded Operators

This multiple use of an operator is called operator overloading.

A single operator symbol has more than one meaning

22. How does short-circuit? 短路求值 7.6 Short-Circuit Evaluation

Definition: A short-circuit evaluation of an expression is one in which the result is determined without evaluation all of the operands and/or operators.

For Boolean operators, the second argument is evaluated only if the first argument does not determine the value of the expression. Example: $a \text{ AND } b$: b is only evaluated if a is false; $a \text{ OR } b$: b is only evaluated if a is true.

23. In what way C's switch statement more different than that of many other languages? (2 marks)

show examples (2 marks) 8.2.2.2 Examples of Multiple Selectors

The switch structure in C language hardly restricts the position of case expressions, which may be a highly complex structure in the switch structure.

Example:

```
switch (number)
```

```
default:
```

```
if (number)
```

```
    case 1: case 3: case 5:
```

```
        printf ("The number is odd"\n);
```

```
    case 2: case 4: case 6:
```

```
        printf ("The number is even"\n);
```

24. In what way is C's for statement more different than that of many other languages?

(Show examples!) 8.3.1.2 The for Statement of the C- Based Languages

The general form of C's for statement:

```
for(initialization_exp ; loop_condition ; increment_exp)
```

```
    loop body; // it can be a single statement / a compound statement / a null statement;
```

Some features:

1. The three expressions in the C's for statement are all omissible.
2. C's for statement does not require counting.
3. The last two loop parameters of the C's for statement are calculated every time the loop is repeated, and the variables in these parameter expressions can be modified in the loop body.

25. 9.5.2 Implementation Models of Parameter Passing

Consider the following program written in C syntax (only syntax - it is not a C code):

```
void swap(int a, int b) {  
    int temp;  
    temp = a;  
    a = b;  
    b = temp;  
}  
  
void main() {  
    int value = 2, list[5] = {1, 3, 5, 7, 9};  
    swap(value, list[0]);  
    swap(list[0], list[1]);  
    swap(value, list[value]);  
}
```

For each of the following parameter-passing methods, what are all of the values of the variables value and list after each of the three calls to swap?

- a. Passed by value
- b. Passed by reference

- a. 2 1 3 5 7 9
 2 1 3 5 7 9
 2 1 3 5 7 9
- b. 1 2 3 4 7 9
 1 3 2 5 7 9
 2 3 1 5 7 9

26. Consider the program below in a hypothetical language which allows global variable and a choice of call by reference or call by value methods of parameter passing.

What will be the output of the program for the two parameter passing mechanisms?

Explain it for pass-by-value and pass-by-reference as well! [4 marks]

```
int i ;  
program main ()  
{  
    int j = 60;  
    i = 50;  
    call f (i, j);  
    print i, j;  
}  
  
procedure f (x, y)  
{  
    i = 100;  
    x = 10;  
    y = y + i ;  
}
```

9.5.2 Implementation Models of Parameter Passing

pass-by-value: i=100, j=60 (执行了 i=100 的定义和函数操作, 但操作完的值不传回)

i is global declared, j is a local variable in main. At first i=50, j=60. When f called value, i and j are passed to it. And i=100 change the global i to 100, x=10 changes the local x from 50 to 10, and y=60+100=160. When returning back to main, i=100 and j=60;

pass-by-reference: i=10, i=70

When f called value, x and y point to the memory location of i and j respectively. So x=10 means x, that is, global i=10, y=y+i=60+10=70. When returning back to main, i will be 100 and j will be 60. When returning back to main, i=10 and j=70;

27. What is the Pass-by-reference model of parameter passing? (describe it) 9.5.2.4 Pass- by- Reference

Explain in your answer the parameter collision problem! (4 points) 9.5.2.2 Pass- by- Result

Pass-by-reference refers to passing the address of the actual parameter directly to the function when calling the function, and the modification of the parameter in the function will also affect the actual parameter.

Parameter collision occurs for the order can be implementation of expressions matter, so different implementations may produce different results.

28. What is an overloaded subprogram? 9.9 Overloaded Subprograms

It is a subprogram which has the same name as other subprograms, but different in number, type order of parameters or return value in the same reference environment.

29. What is a generic subprogram?

Explain it detailly (2 marks) and show an example (2 marks). 9.10 Generic Subprograms

Parameter polymorphism means that no specific types are specified during declarations and definitions, but these types are used as parameters, so that the definition is applicable to various specific types.

参数多态是指在声明和定义时不指定具体的类型，而是将这些类型作为参数使用，从而使定义适用于各种具体的类型

And a generic subprogram is a subprogram which have parametric polymorphism.

泛型子程序是具有参数多态性的子程序

Example: Generic functions in C++ is template function

```
template <typename T>
```

```
class List {
```

```
    // Class contents.
```

```
};
```

```
List<Number> list_of_numbers;
```

```
List<Car> list_of_cars;
```

30. What is a coroutine? 9.13 Coroutines

it is a kind of computer program components that generalize subroutines for non-preemptive multitasking, by allowing execution to be suspended and resumed

它是一种计算机程序组件，通过允许暂停和恢复执行来概括非抢占式多任务处理的子程序